

 **Note**

학습 목표 이 챕터에서는 아래 내용을 목표로 합니다.

- LLM의 본질과 한계를 명확히 이해하고 설명할 수 있다
- Agent가 왜 필요한지, 어떻게 동작하는지 파악할 수 있다
- DAG 문제와 Context Engineering의 관계를 설명할 수 있다
- 주요 Agent 프레임워크의 역할과 선택 기준을 안다

들어가며: AI의 변곡점에서

GPT-5.1, Claude 4.5 Sonnet, Gemini 2.0 Flash... 최신 LLM은 이미 기본적인 Agent 기능까지 내장하고 있습니다. 하지만 현장에서는 여전히 의문이 생깁니다:

"AI가 단순 질의응답을 넘어, 복잡한 업무를 자율적으로 처리하려면?"

정답은 **Agent**입니다. 단순히 '답변'만 생성하는 LLM을 넘어, '계획-실행-검증'의 순환을 반복하며 목표를 달성하는 시스템이 필요합니다.

최신 모델들은 이미 Agent 기능을 기본 탑재하고 있지만, 진짜 난제는 따로 있습니다. 바로 복잡한 워크플로우 설계(DAG 문제)와 이를 해결하는 **Context Engineering**입니다.

 1단계: LLM 기초 이해하기

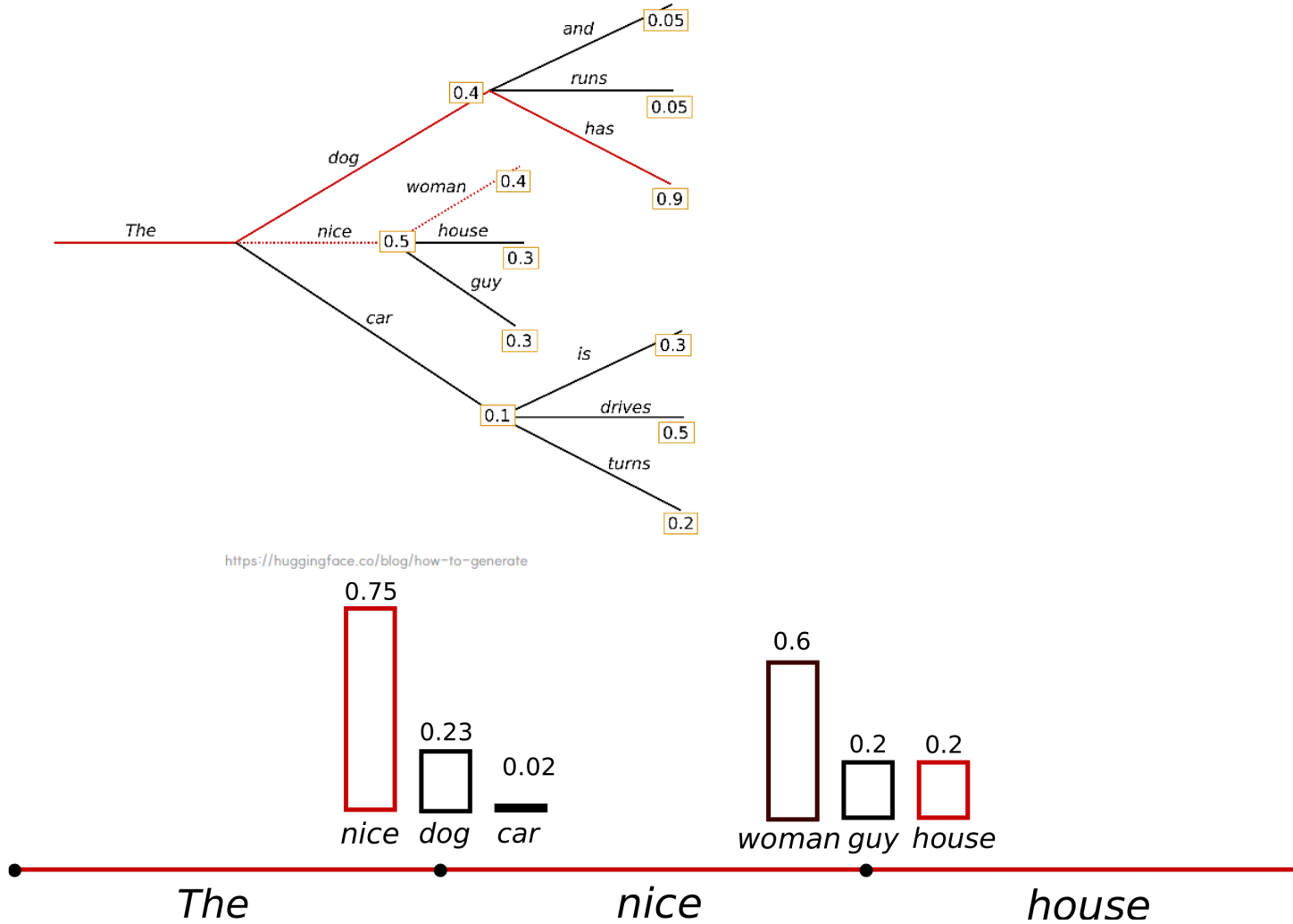
1.1 LLM은 어떻게 작동하는가?

LLM의 핵심은 단순합니다: "다음에 올 단어를 예측하는 확률 모델"

```
def llm_core(input_tokens, model_weights):  
    # 입력 토큰 시퀀스 → 다음 토큰 확률 분포 계산  
    next_token_probabilities = compute_probabilities(input_tokens, model_weights)  
  
    # 확률에 따라 다음 토큰 선택  
    next_token = sample_from_distribution(next_token_probabilities)  
  
    return next_token
```

이 간단한 과정의 반복에서 **창발적 능력(Emergent Abilities)** 이 나타납니다:

- 산술 연산
- 논리적 추론 (Chain-of-Thought)
- 코드 생성



텍스트 생성 방법: Transformers를 이용한 언어 생성을 위한 다양한 디코딩 방법

1.2 네 가지 시스템적 제약

LLM은 강력하지만, 4가지 근본적 한계가 있습니다:

제약	설명	영향
Stateless	이전 대화를 기억 못함	매 요청마다 컨텍스트 재전달 필요
Context Window	처리 가능한 정보량 제한	긴 문서/대화는 분할 필요
Hallucination	그럴듯한 거짓 정보 생성	외부 검증/RAG 필수
Knowledge Cutoff	학습 이후 정보 모름	Tool Use로 실시간 정보 보완

예시: Stateless 문제

```
# ❌ 잘못된 기대
conversation_1 = llm("제 이름은 김철수입니다.")
conversation_2 = llm("제 이름이 뭐라고 했죠?") # 답할 수 없음!

# ✅ 올바른 접근
history = [
    ("user", "제 이름은 김철수입니다."),
    ("assistant", "안녕하세요 김철수님!")
]
response = llm(history + [("user", "제 이름이 뭐라고 했죠?")]) # 이제 가능!
```

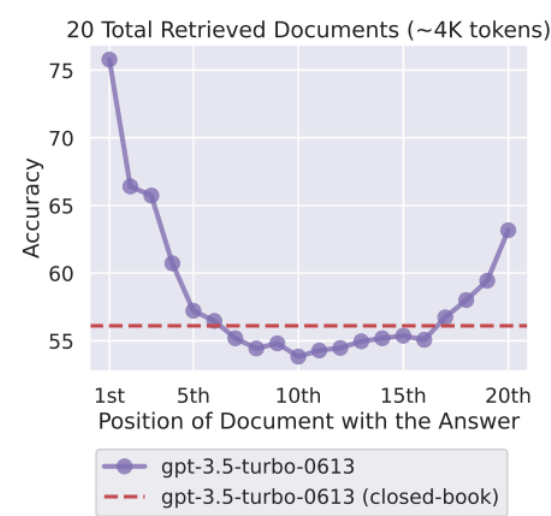
최신 모델의 Context Window (2025년 11월 기준)

모델	입력 토큰 한도	비고	제공사
GPT-5.1	272,000 (입력) + 128,000 (출력)	총 400K	OpenAI

모델	입력 토큰 한도	비고	제공사
Claude 4.5 Sonnet	200,000 (기본)	베타에서 1M까지 확장 가능	Anthropic
Gemini 2.0 Flash	1,048,576	약 1M	Google

하지만 두 가지 문제는 여전합니다:

- 1. 절대적 용량 한계: 대규모 코드베이스는 여전히 단일 요청 불가
- 2. 'Lost in the Middle' 현상: 중간 정보는 잘 놓침



1.3 핵심 통찰: Context가 전부다

LLM = 시험장의 학생

- 시험지(Context Window)의 정보만 사용 가능
- 다른 시험(Stateless)의 내용은 모름
- 모르면 추측(Hallucination)
- 시험 후 세상(Knowledge Cutoff) 소식 모름

🔗 더 알아보기: Context Engineering은 '목발'인가?

Open AI의 Noam Brown은 RAG, ReAct, CoT 같은 현재 기법들을 LLM의 추론 능력 부족을 보완하는 '목발(**crutch**)'이라고 표현했습니다. 모델이 발전하면 이런 기법 의존도가 줄어들 것이지만, 현재로서는 필수적입니다.

- [인터뷰 리뷰에 대한 유튜브](#)

✅ 체크포인트: 1단계

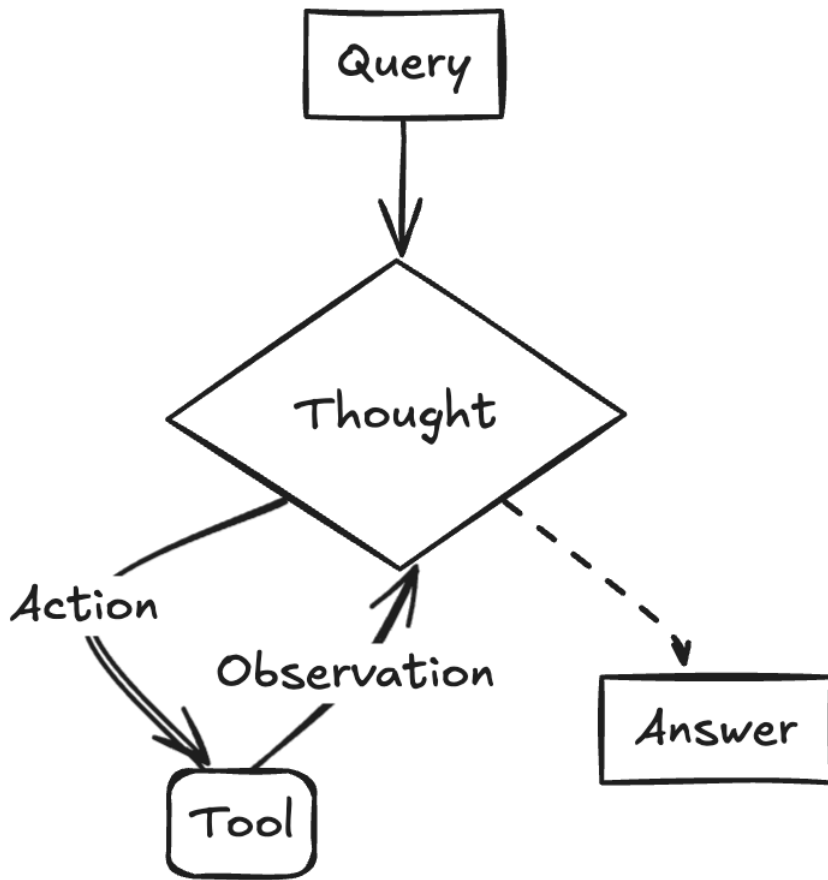
- ☐ LLM의 토큰 예측 메커니즘을 설명할 수 있다
- ☐ 4가지 시스템 제약(Stateless, Context Window, Hallucination, Knowledge Cutoff)을 이해한다
- ☐ Context Window 크기와 한계를 안다
- ☐ 간단한 코드로 대화 기록을 관리하는 방법을 구현할 수 있다

🤖 2단계: Agent 패러다임 이해하기

2.1 ReAct: 생각하고 행동하기

2022년 구글의 **ReAct(Reason + Act)** 논문은 현대 Agent의 기틀을 마련했습니다.

핵심 루프: Thought(생각) → Action(행동) → Observation(관찰)



전통적 LLM vs ReAct 비교

```

# ❌ 전통적 LLM
response = llm("서울의 현재 날씨는?")
# → "죄송하지만, 실시간 정보에 접근할 수 없습니다..."

# ✅ ReAct 패턴
# 1. Thought: 문제 분석
thought = llm("날씨 정보를 얻으려면 '날씨 API'를 호출해야겠다.")

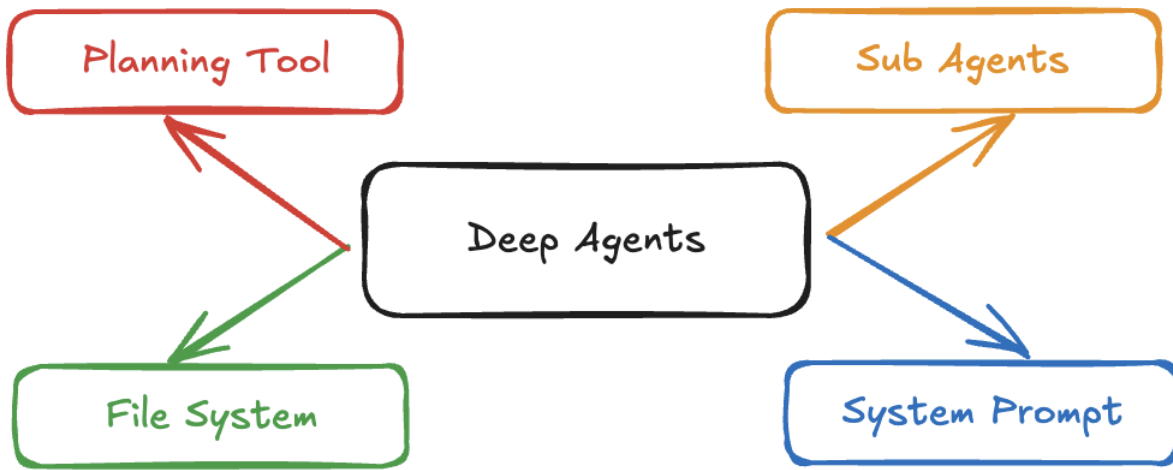
# 2. Action: 도구 실행
action = "weather_api.get('서울')"
observation = execute_tool(action) # → {"온도": 25, "상태": "맑음"}

# 3. Observation: 결과 반영
final_response = llm(f"""
이전 생각: {thought}
관찰 결과: {observation}
→ 사용자에게 최종 답변 생성 또는 도구 한번 더 호출
""")
# → "현재 서울의 날씨는 25도이며, 맑은 상태입니다."
  
```

2.2 Agent의 4가지 핵심 모듈

현대 Agent는 4개 모듈의 상호작용으로 구성됩니다. 이는 다양한 연구와 산업 표준에서 공통적으로 제시하는 구조입니다:

모듈	역할	예시
Profiling	Agent 정체성 정의	"당신은 친절한 AI 비서입니다"
Memory	경험과 맥락 유지	대화 기록, 학습한 정보 저장
Planning	목표 달성 계획 수립	작업 분해, 도구 선택
Action	계획 실행	API 호출, 결과 전달



Info

- 삼성SDS 인사이트 - AI 에이전트 핵심 능력 분석
- Enhans.ai - 자동화 웹 에이전트 연구 동향

일부 문헌에서는 Profiling 대신 **Perception(지각)**을 사용하기도 하지만, 핵심 구조는 동일합니다.

메모리 구조:

- 단기 메모리(Short-term): 현재 세션 정보
- 장기 메모리(Long-term): 사용자 선호도, 학습 내용
- 에피소드 메모리(Episodic): 특정 시간·장소와 연결된 개인적 경험 (예: "어제 사용자가 날씨 API 호출했음")
- 세맨틱 메모리(Semantic): 시간·장소와 무관한 일반적 지식·사실 (예: "날씨 API는 OpenWeatherMap 사용")

✅ 체크포인트: 2단계

- ☐ ReAct 패턴의 3단계(Thought → Action → Observation)를 설명할 수 있다
- ☐ Agent의 4가지 모듈(Profiling, Memory, Planning, Action)을 이해한다
- ☐ 단기/장기 메모리의 차이를 안다
- ☐ 간단한 ReAct 패턴 Agent를 구현할 수 있다

실습 예제:

- 날씨 API를 연결한 ReAct 패턴 Agent 만들기
- 대화 기록을 저장하고 참조하는 메모리 시스템 구현

⚙️ 3단계: DAG 문제 이해하기

3.1 이상 vs 현실

Agent 작업은 보통 이렇게 구성됩니다:

정보 수집 → 분석 → 보고서 초안 → 검토 → 최종본

이는 **DAG(Directed Acyclic Graph, 방향성 비순환 그래프)** 구조입니다.

하지만 현실은 다릅니다:

1. 재시도: API 호출 실패 시 다시 시도
2. 조건부 분기: 품질 낮으면 다른 도구 사용
3. 인간 개입: 중간 결과 검토 후 이전 단계로 회귀

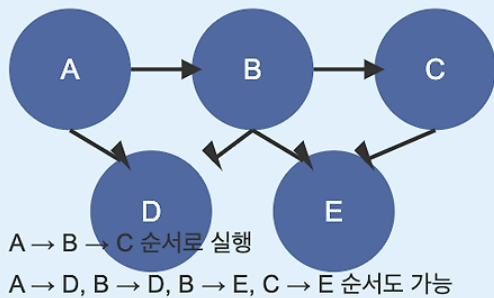
→ 모두 **순환(Cycle)**이 필요합니다

DAG (Directed Acyclic Graph) 개념

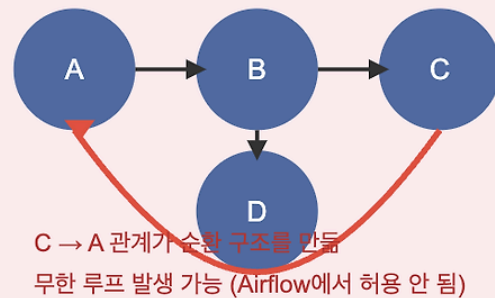
DAG는 작업 흐름을 방향성 있는 비순환 그래프로 표현합니다.

- 방향성(Directed): 각 작업 간의 관계가 방향을 가집니다.
- 비순환(Acyclic): 사이클이 발생하지 않습니다.
- 그래프(Graph): 노드(작업)와 엣지(의존성)로 구성됩니다.

올바른 DAG 구조



잘못된 DAG 구조 (순환)



Airflow DAG는 방향성 있는 비순환 그래프로, 작업 간의 명확한 의존성을 정의합니다

3.2 실전 사례: 투자 자문 Agent

```
class InvestmentAdvisorAgent:
    def analyze_and_recommend(self, user_profile):
        # 1단계: 데이터 수집
        market_data = self.collect_market_data()

        # 2단계: 데이터 검증 → 불충분하면 1단계로 돌아가기 필요
        if not self.is_data_sufficient(market_data):
            additional_data = self.collect_additional_data()
            market_data.update(additional_data)

        # 3단계: 추천안 생성
        recommendations = self.generate_recommendations(market_data, user_profile)

        # 4단계: 리스크 검증 → 높으면 3단계로 돌아가기 필요
        risk_score = self.assess_risk(recommendations)
        if risk_score > user_profile.risk_threshold:
            # 단순 DAG로는 이 복잡한 제어 흐름을 표현 불가!
            ...
```

✅ 체크포인트: 3단계

- ☐ DAG의 정의와 한계를 설명할 수 있다
- ☐ 재시도, 분기, 순환이 왜 필요한지 안다
- ☐ 실제 워크플로우에서 DAG 문제가 발생하는 시나리오를 식별할 수 있다

🧩 통합 실습: Agent 워크플로우 설계하기

목표: 1~3단계에서 배운 개념(LLM 한계, Agent 모듈, DAG 문제)을 통합해 실제 문제를 해결하는 Agent 워크플로우를 Excalidraw로 시각화

사용자가 "배송이 안 왔어요"라고 문의
→ Agent가 주문 조회 → 배송 상태 확인 → 답변 생성
→ 만약 DB 조회 실패? 재시도 필요
→ 만약 배송 지연? 보상 정책 확인 후 재안내

🔗 반드시 포함해야 할 요소

1단계 지식 (LLM 한계 해결):

- Stateless 해결: 대화 기록 저장소 표시
- Knowledge Cutoff 해결: 실시간 주문 DB 연결 표시

2단계 지식 (Agent 4가지 모듈):

- **Profiling**: Agent 역할 정의 (예: "친절한 고객 지원 AI")
- **Memory**: 단기 메모리 (현재 주문 정보) 표시
- **Planning**: 의사결정 로직 (다이아몬드 ◇ 기호로 분기)
- **Action**: 실제 도구 호출 (DB 조회, 보상 정책 API 등)

3단계 지식 (DAG 문제 해결):

- 재시도 로직: DB 조회 실패 시 3번까지 재시도
- 조건부 분기: 배송 상태에 따라 다른 응답 경로
- 순환 구조: 품질 검증 실패 시 이전 단계로 회귀

🌟 4단계: Context Engineering 마스터하기

4.1 정의

Context Engineering이란?

Agent의 작업 흐름 전반에 걸쳐, 목표 달성에 필요한 최적의 정보를, 가장 적절한 시점에, 올바른 형식으로 LLM에게 동적으로 공급하는 모든 기술과 전략

단순 Prompt Engineering ≠ Context Engineering

- **Prompt**: 한 번의 입력 최적화
- **Context**: Agent 전체 흐름의 '작업 기억' 설계

4.2 핵심 기술 3가지

1) RAG (Retrieval-Augmented Generation)

외부 지식 베이스 참조로 환각 줄이고 최신 정보 반영

```
def rag_enhanced_llm(query):  
    # 1. Vector DB에서 관련 문서 검색  
    relevant_docs = vector_db.similarity_search(query)  
  
    # 2. 검색 정보를 컨텍스트에 포함  
    context = f"""  
    [관련 정보]  
    {relevant_docs}  
  
    [질문]  
    {query}  
  
    → 위 정보를 바탕으로 정확히 답변하세요.  
    """  
    return llm(context)
```

주요 벡터 DB:

- **ChromaDB**: 간단한 로컬 테스트용
- **Pinecone**: 클라우드 완전 관리형

- **Weaviate:** 오픈소스, 하이브리드 검색

2) 동적 메모리 관리

대화 기록을 요약하거나 중요 정보만 선별해 컨텍스트에 포함

예시 도구:

- **Mem0** - AI 앱용 메모리 레이어

3) 상태 인식 포매팅

Agent 현재 상태에 따라 정보 형식 최적화

상태	컨텍스트에 포함할 정보
계획 수립 중	목표, 사용 가능 도구, 제약 조건
코드 실행 중	실행 환경, 의존성, 보안 정책
디버깅 중	에러 로그, 스택 트레이스, 이전 시도

4.3 Context Engineering이 DAG 문제를 해결하는 원리

DAG 구조를 직접 수정하는 대신, **컨텍스트를 동적 제어**:

- **상태 보존:** "현재 2번째 재시도 중" 같은 정보를 컨텍스트에 명시
- **논리적 순환:** 컨텍스트에 "품질 미달, 분석 단계로 돌아가 다른 방법 시도" 지침 제공

→ 물리적 순환 없이도 논리적 순환 구현!

✅ 체크포인트: 4단계

- ☐ Context Engineering과 Prompt Engineering의 차이를 설명할 수 있다
- ☐ RAG의 작동 원리를 이해하고 구현할 수 있다
- ☐ 동적 메모리 관리의 필요성을 안다
- ☐ 상태에 따른 컨텍스트 포매팅 전략을 수립할 수 있다

실습 예제:

- 간단한 RAG 시스템 구축 (Vector DB + LLM)
- 대화 요약 및 압축 메커니즘 구현
- 상태별 프롬프트 템플릿 설계

🔥 5단계: 프레임워크 선택하기

5.1 왜 프레임워크가 필요한가?

Agent는 수많은 외부 도구와 상호작용합니다. 각각 다른 사용법이라면 복잡도가 기하급수적 증가!

프레임워크 = LLM 연동 + 메모리 관리 + 도구 연결 + 실행 루프

→ 개발자는 핵심 비즈니스 로직에만 집중 가능

5.2 프레임워크 비교

프레임워크	핵심 강점	주요 해결 과제	적합한 사용 사례
LangFlow	시각적 UI, No-Code	빠른 프로토타이핑, 아이디어 검증	PoC, 비개발자의 Agent 구축
LangGraph	상태 관리, 순환 지원	DAG 문제 직접 해결	복잡한 상태를 가진 자율 Agent
LangChain	풍부한 통합 생태계	기본 Agent 및 RAG 파이프라인	표준적인 LLM 애플리케이션
LangFuse	관찰 가능성, 디버깅	프로덕션 모니터링 및 분석	운영 및 유지보수

5.3 LangGraph가 특별한 이유

LangGraph = DAG 문제를 위해 특별히 설계된 프레임워크

핵심 기능:

- 상태 관리: 각 단계를 State 변경 노드로 정의
- 조건부 엣지: 상태에 따라 다음 노드 동적 결정
- 순환 지원: 명시적 순환 구조 구현 가능

LangGraph 코드 예시

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

# 1. Agent 상태 정의
class AgentState(TypedDict):
    messages: List[str]
    quality_score: float

workflow = StateGraph(AgentState)

# 2. 노드(작업 단위) 정의
workflow.add_node("analyze", analyze_node) # 분석 노드
workflow.add_node("review", review_node) # 검토 노드

# 3. 시작점 설정
workflow.set_entry_point("analyze")

# 4. 조건부 엣지 → 순환 가능!
def decide_next_step(state: AgentState):
    if state["quality_score"] < 0.8:
        return "analyze" # 품질 미달 → 다시 분석
    else:
        return END # 통과 → 종료

workflow.add_conditional_edges("review", decide_next_step)
workflow.add_edge("analyze", "review")

# 5. 그래프 컴파일
app = workflow.compile()
```

5.4 MCP (Model Context Protocol)

MCP = 모델과 컨텍스트 간 표준화된 통신 프로토콜

Anthropic이 주도하는 표준으로, 다음을 명확한 태그로 구분:

- 시스템 역할
- 사용자 요청
- 메모리
- 도구 호출
- 코드 스니펫

→ 모델이 혼동 없이 완전한 컨텍스트 수신 가능

MCP 활용:

- `/messages` : 대화 기록 엔드포인트
- `/state` : 현재 상태 조회
- `/health` : 서버 상태 확인

- 4가지 주요 프레임워크(LangFlow, LangGraph, LangChain, LangFuse)의 차이를 설명할 수 있다
- LangGraph의 상태 관리와 조건부 엣지 개념을 이해한다
- 프로젝트 요구사항에 맞는 프레임워크를 선택할 수 있다
- MCP 프로토콜의 역할을 안다
- LangGraph로 순환 구조를 구현할 수 있다

실습 예제:

- LangChain으로 기본 RAG 파이프라인 구축
- LangGraph로 품질 점수 기반 순환 워크플로우 만들기
- 각 프레임워크로 동일한 Agent 구현 후 비교

📺 6단계: 프로덕션 준비하기

6.1 모델 파라미터 이해하기

파라미터	설명	권장 값
Temperature	창의성 조절 (0~2)	0.0: 결정적, 0.7: 균형, 1.5: 창의적
Top-p	Nucleus Sampling (0~1)	0.9: 안정적, 0.5: 보수적
Max Tokens	최대 출력 길이	작업에 따라 조정
Frequency Penalty	반복 단어 억제 (-2~2)	0.5: 약간 억제
Presence Penalty	새 주제 촉진 (-2~2)	0.3: 약간 촉진

Model is deterministic.
Always picks the word with
the highest probability

encourages more creativity
by allowing the model to
consider words with lower
probabilities

Strongly biases the model
towards lower-probability
words, leading to highly
unusual or unexpected
outputs.

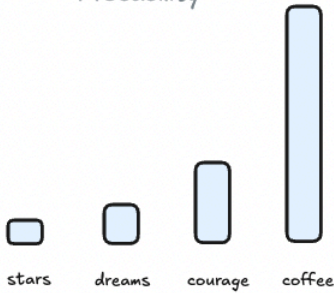
0 ————— Temperature ————— 2

"A cup of coffee."

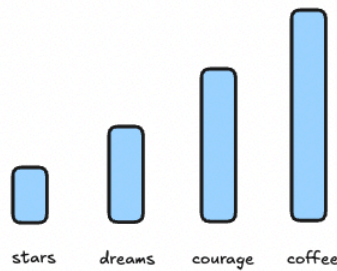
"A cup of courage."

"A cup of stars."

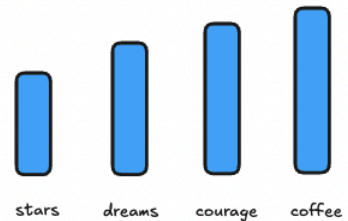
Probability



Probability



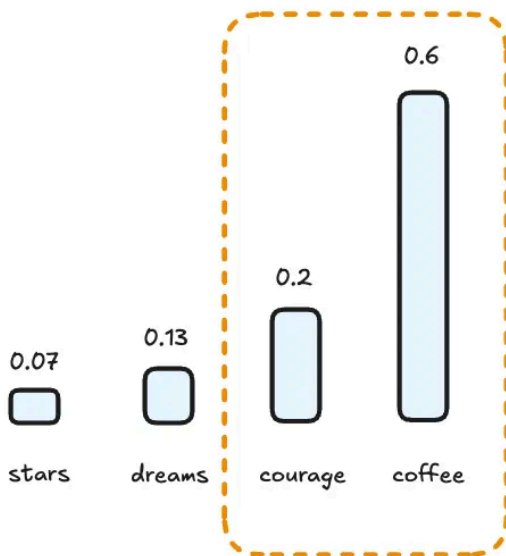
Probability



 www.bighummingbird.com

Top-K

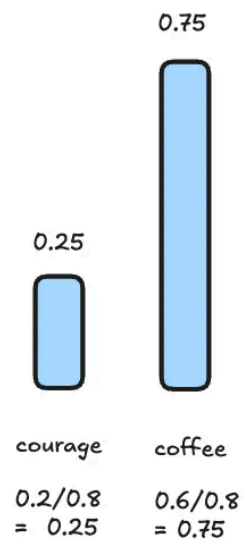
Original Distribution



$$0.6 + 0.2 = 0.8$$

Top-k with $k = 2$
means picking top 2

New Distribution

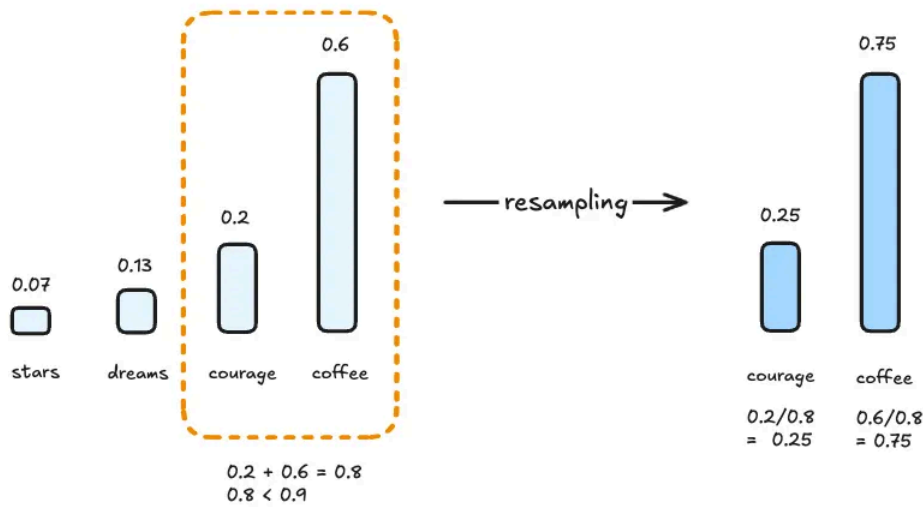


We now sample from this
new distribution

Top-p

Original Distribution

New Distribution



Top-p with $p = 0.9$
means picking all the top tokens
whose likelihoods add up to 0.9
Ignore all others

We now sample from this
new distribution

6.2 도구 통합 (Tool Use)

Agent가 사용할 수 있는 도구 유형:

도구 유형	예시	활용
웹 검색	Google, Bing API	실시간 정보 수집
코드 실행	REPL, Sandbox	계산, 데이터 처리
데이터베이스	SQL, NoSQL	구조화 데이터 조회
API 호출	REST, GraphQL	외부 서비스 연동
파일 시스템	Read/Write	로컬 파일 관리
이메일/메신저	SMTP, Slack API	알림 전송

도구 정의 예시 (OpenAI Function Calling):

```
tools = [  
  {  
    "type": "function",  
    "function": {  
      "name": "get_weather",  
      "description": "특정 도시의 현재 날씨 정보를 가져옵니다",  
      "parameters": {  
        "type": "object",  
        "properties": {  
          "city": {  
            "type": "string",  
            "description": "도시 이름 (예: 서울, 부산)"  
          },  
          "unit": {  
            "type": "string",  
            "enum": ["celsius", "fahrenheit"],  
            "description": "온도 단위"  
          }  
        }  
      }  
    },  
    "required": ["city"]  
  }  
]
```

```
}
}
}
]
```

6.3 모니터링 및 평가

LangFuse를 활용한 관찰 가능성:

- 토큰 사용량 추적
- 응답 시간 모니터링
- 에러율 분석
- A/B 테스트

🎓 핵심 요약

우리가 배운 것

단계	핵심 개념	왜 중요한가
1. LLM 기초	Context 의존성, 4가지 제약	LLM의 한계를 이해해야 Agent 필요성을 안다
2. Agent 패러다임	ReAct 루프, 4가지 모듈	LLM에 '행동' 능력을 부여하는 핵심 구조
3. DAG 문제	순환의 필요성	현실의 복잡한 워크플로우는 직선형이 아님
4. Context Engineering	RAG, 동적 메모리, 상태 인식	DAG 문제를 컨텍스트 제어로 해결
5. 프레임워크	LangGraph의 상태 관리	복잡한 구현을 추상화하고 생산성 향상
6. 프로덕션	모델 파라미터, 도구 통합	실전 배포를 위한 최적화와 모니터링

📖 참고 자료

필수 읽을거리

- [roadmap.sh/ai-agents](#) - AI Agent 로드맵
- [Anthropic's Model Context Protocol](#) - MCP 공식 문서
- [LangGraph Documentation](#) - LangGraph 가이드
- [Mem0](#) - AI 메모리 레이어

논문

- [ReAct: Synergizing Reasoning and Acting in Language Models](#) (2022)
- [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#) (2020)
- [Chain-of-Thought Prompting Elicits Reasoning in LLMs](#) (2022)

커뮤니티

- LangChain Discord
- r/LocalLLaMA
- Hugging Face Forums